

Sorting

Sorting

- Sorting refers to arranging data in a particular order (ascending (increasing) or descending (decreasing) order).

4	2	1	- 1	8	0
---	---	---	-----	---	---

Unsorted Array

-1	0	1	2	4	8
----	---	---	---	---	---

Array sorted in increasing order

8	4	2	1	0	-1
---	---	---	---	---	----

Array sorted in descending order

Types of Sorting

- There are two main types of sorting.

1. Internal sorting.
2. External sorting.

1. **Internal sorting:** Internal sorting means that all the data that is to be sorted is stored in main memory.

Cont...

2. **External sorting:** External sorting means that the data is stored outside main memory (like on disk) and only loaded into main memory in small chunks.
- External sorting is usually applied in cases when data can't fit into main memory entirely.

Difference Between Internal And External Sorting

Internal Sorting	External Sorting
Data is sorted in main memory	Data is sorted using external devices such as disks or tapes
Not suitable for large data sets	Ideal for sorting large amount of data
Faster than external sorting	Slower than internal sorting
More efficient for small datasets	More efficient for larger datasets
Uses less resources	Requires more resources
Example: Bubble sort, Insertion Sort, Selection sort, Quick sort, Heap sort, etc.	Example: Merge sort etc.

Bubble Sort

- **Bubble Sort** is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in the wrong order.
- It is called bubble sort because the movement of array elements is just like the movement of air bubbles in the water.
- This algorithm is not suitable for large data sets

Example

- Write steps during sorting the sequence 5, 1, 6, 2, 4 and 3 using bubble sort

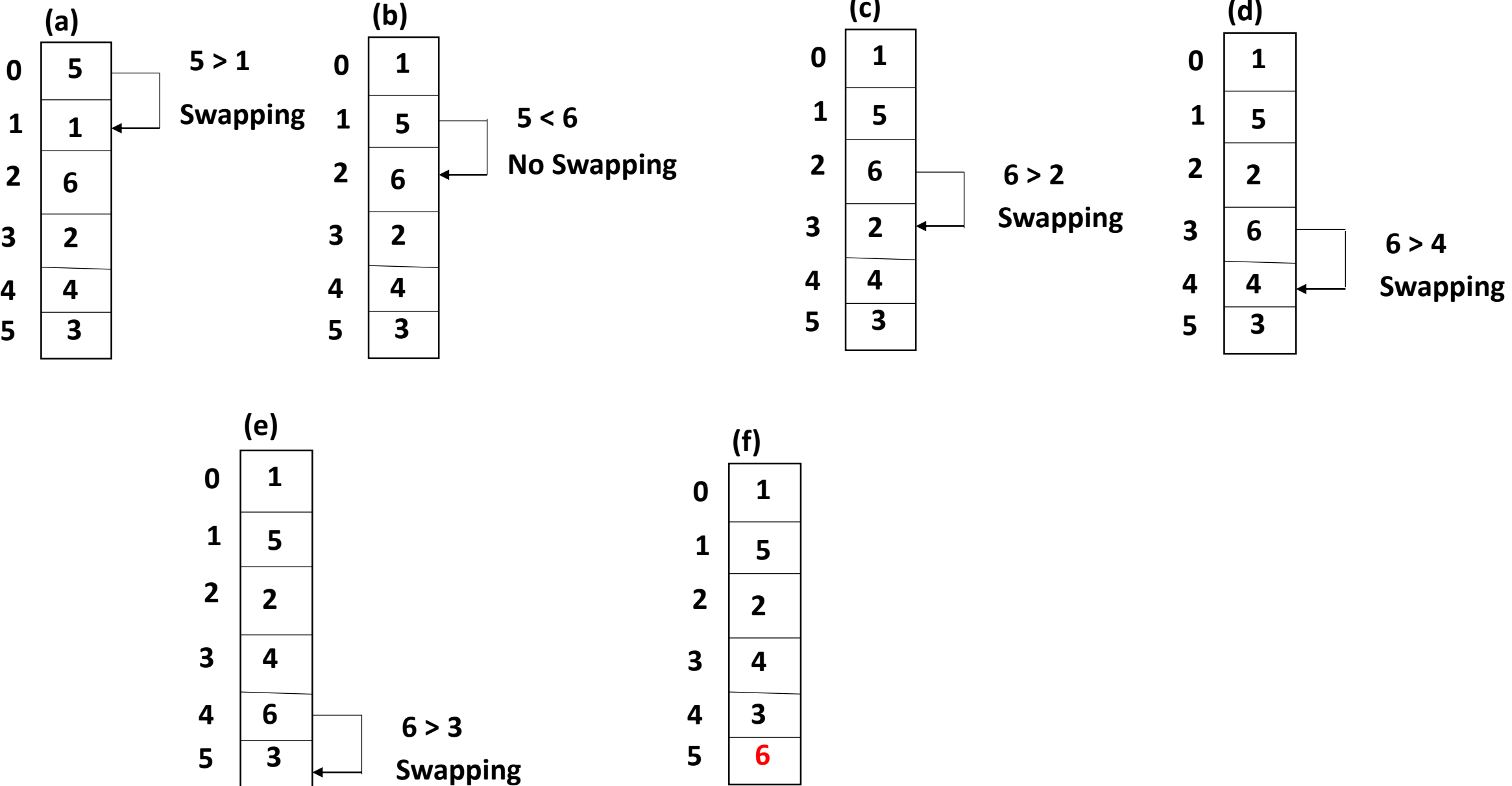
Cont...

Ans:

0	1	2	3	4	5
5	1	6	2	4	3

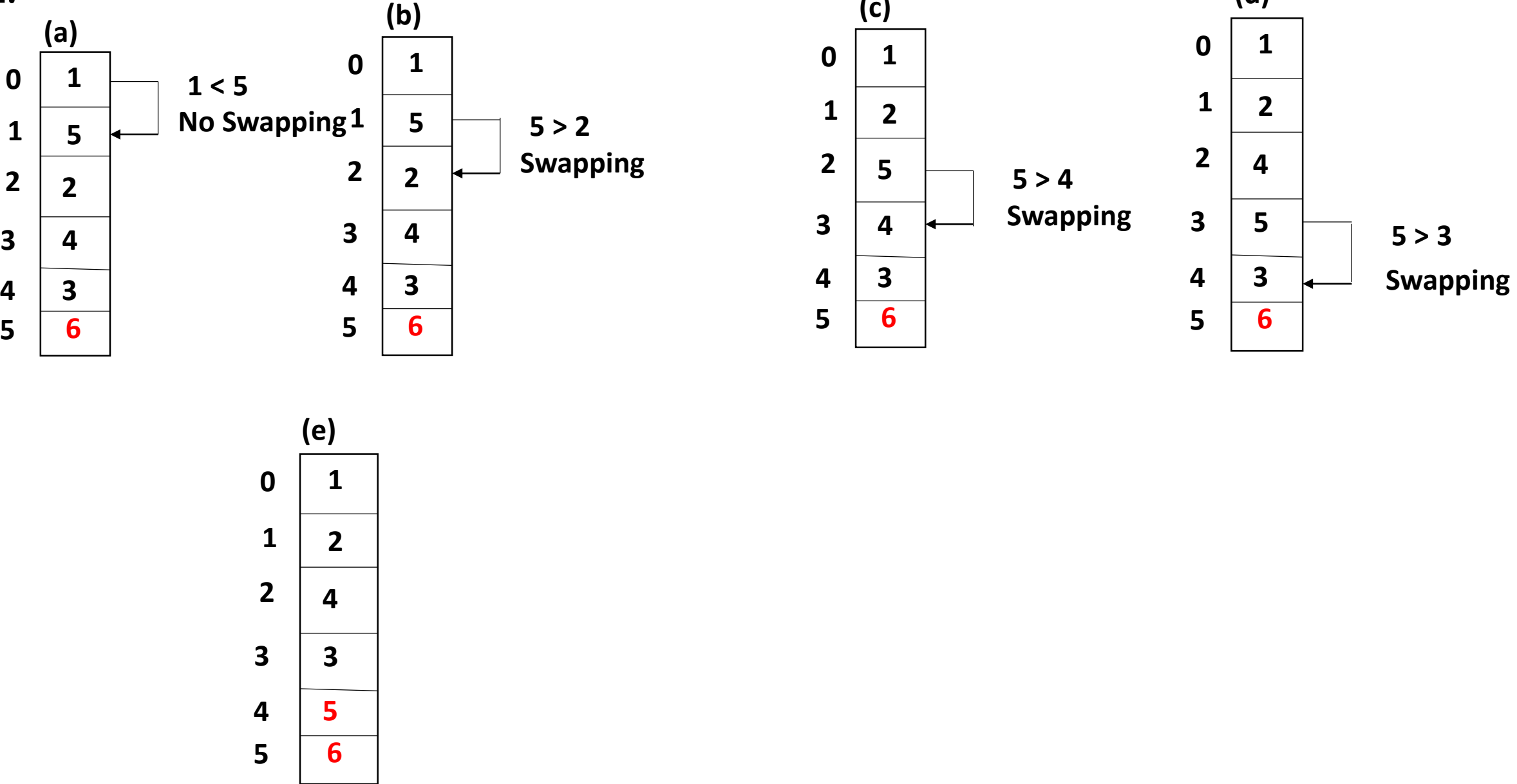
Cont...

Step 1:



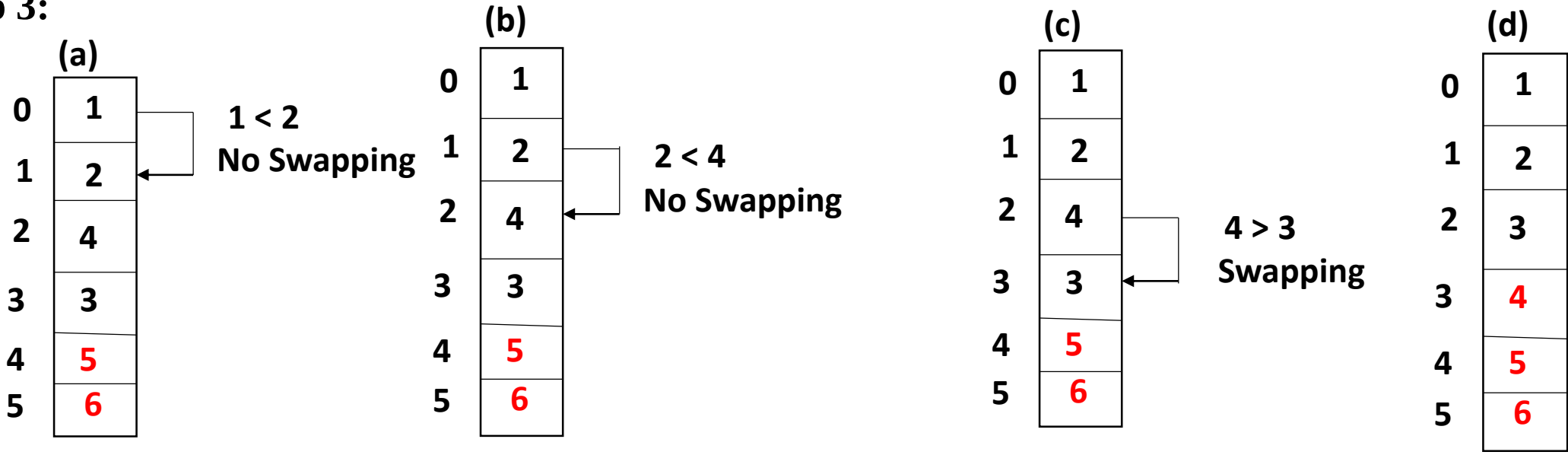
Cont...

Step 2:



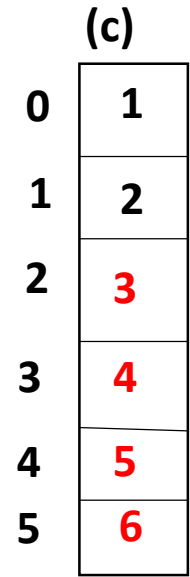
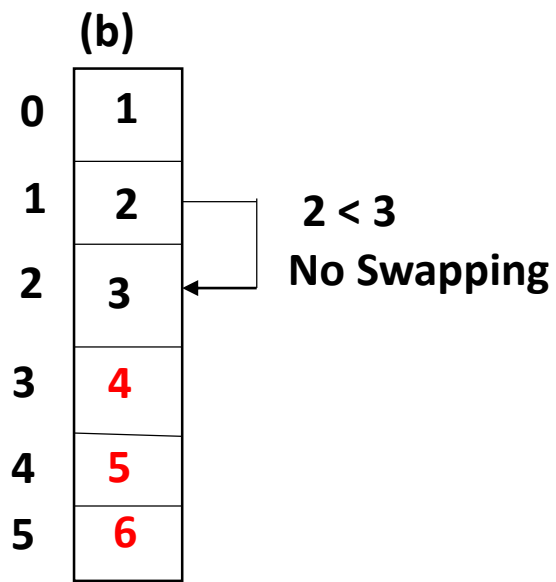
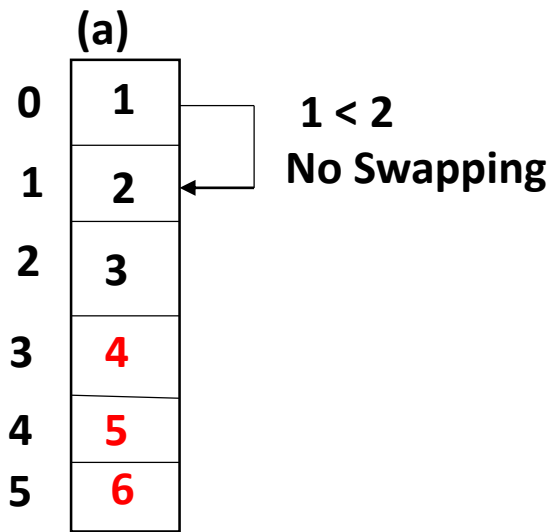
Cont...

Step 3:



Cont...

Step 4:



Cont...

Step 5: (a)

0	1
1	2
2	3
3	4
4	5
5	6



$1 < 2$

No Swapping

(b)

0	1
1	2
2	3
3	4
4	5
5	6

Program

```
#include<stdio.h>
void main()
{
    int a[6], n, i, j;
    printf("enter total no of elements");
    scanf("%d", &n);
    printf("enter elements");
    for(i=0; i<=n-1; i++)
    {
        scanf("%d", &a[i]);
    }
}
```

```
for(i=0; i<=n-2; i++)
{
    for(j=0; j<=n-2-i; j++)
    {
        if(a[j]>a[j+1])
        {
            int temp ;
            temp = a[j];
            a[j] = a[j+1];
            a[j+1] = temp;
        }
    }
}
for(i=0;i<=n-1;i++)
{
    printf("%d",a[i]);
}
}
```

Insertion Sort

- **Insertion sort** is an algorithm used to sort a collection of elements in ascending or descending order.
- The basic idea behind the algorithm is to divide the list into two parts: a sorted part and an unsorted part.
- Initially, the sorted part contains only the first element of the list, while the rest of the list is in the unsorted part.

Cont...

- The algorithm then iterates through each element in the unsorted part, picking one at a time, and inserts it into its correct position in the sorted part.

Example

- Write steps during sorting the sequence 5, 1, 6, 2, 4 and 3 using insertion sort

Cont...

Ans:

0	1	2	3	4	5
5	1	6	2	4	3

--

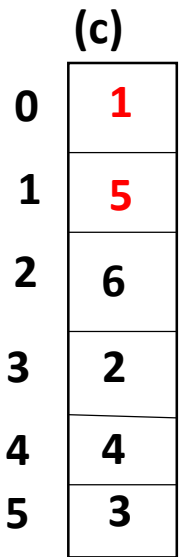
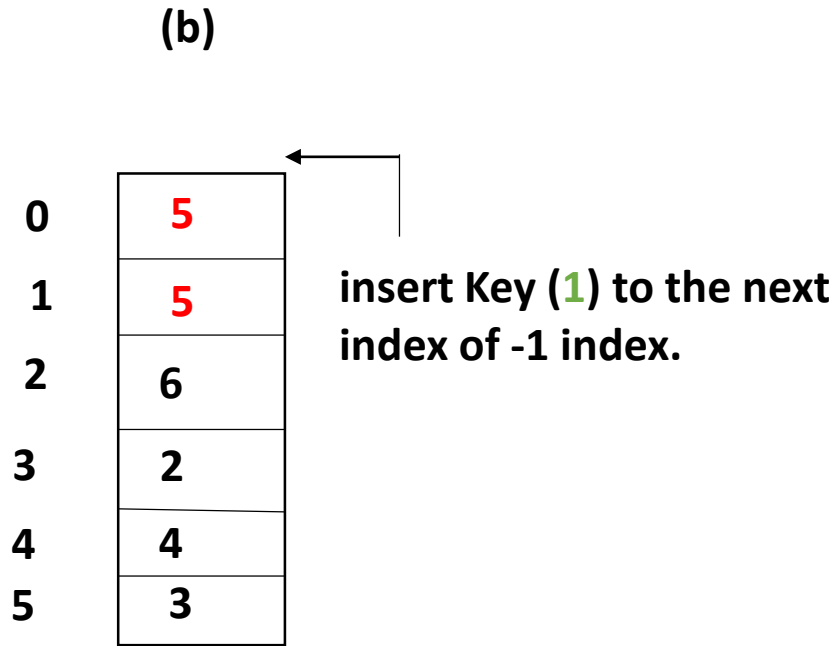
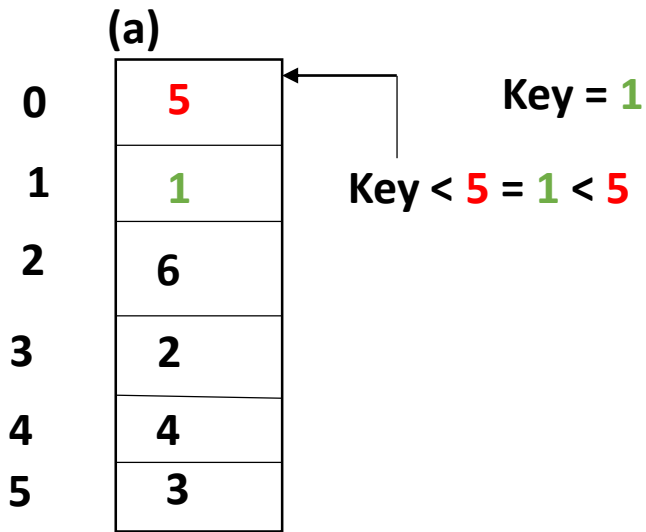
**Sorted
Array**

--

Un-Sorted Array

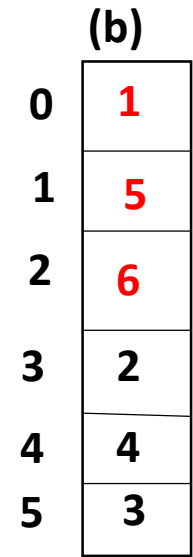
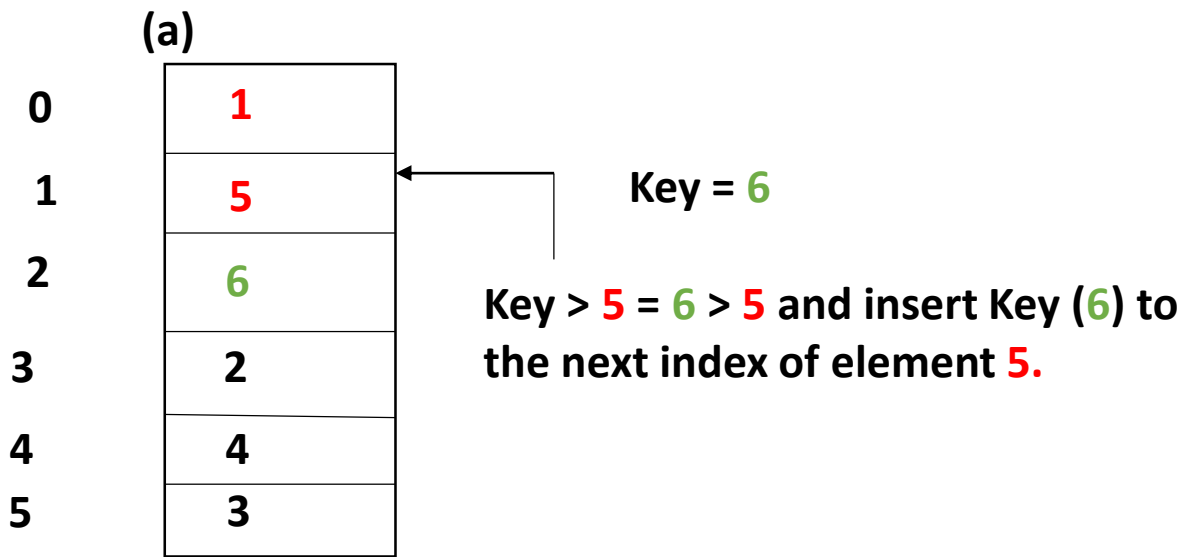
Cont...

Step 1:



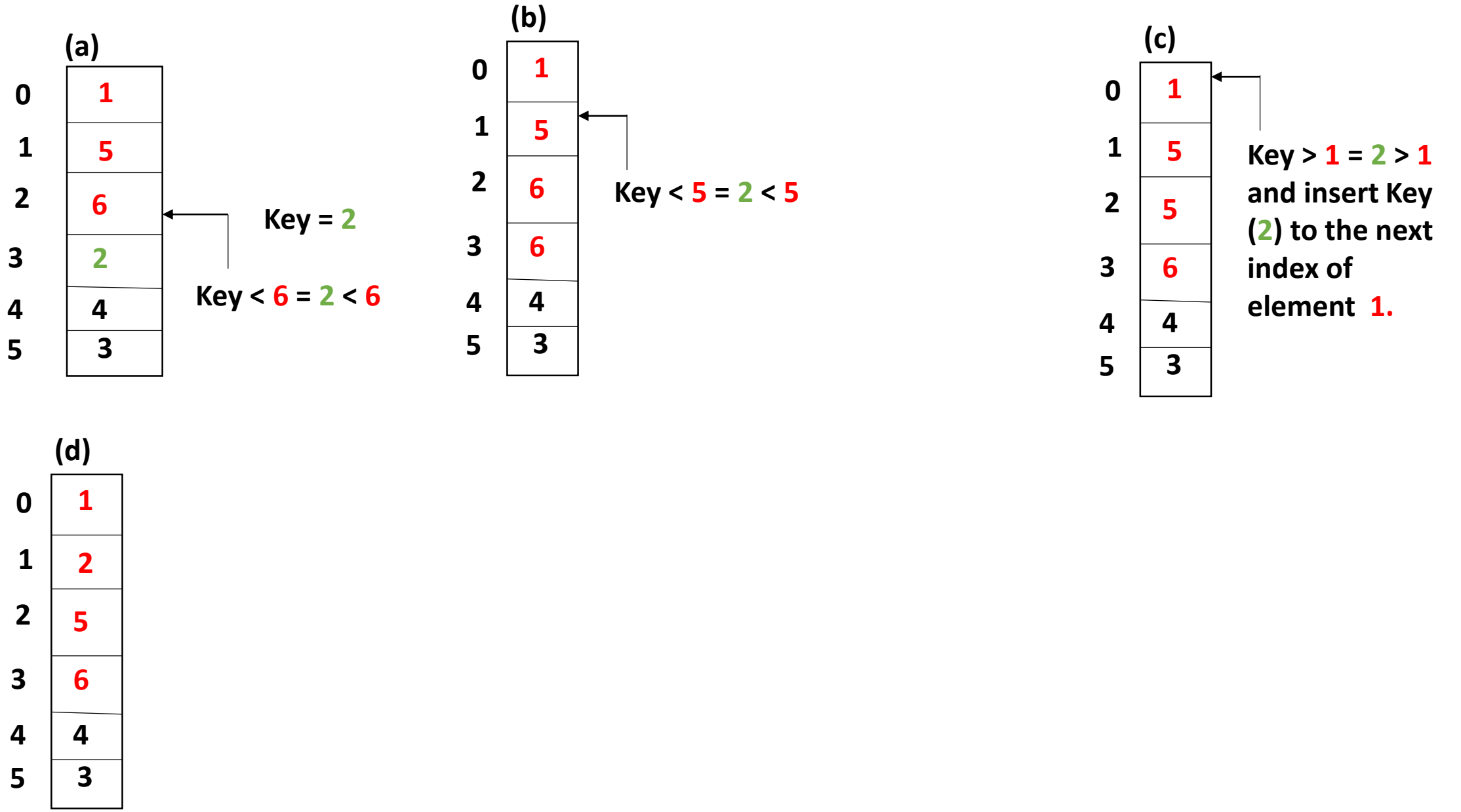
Cont...

Step 2:



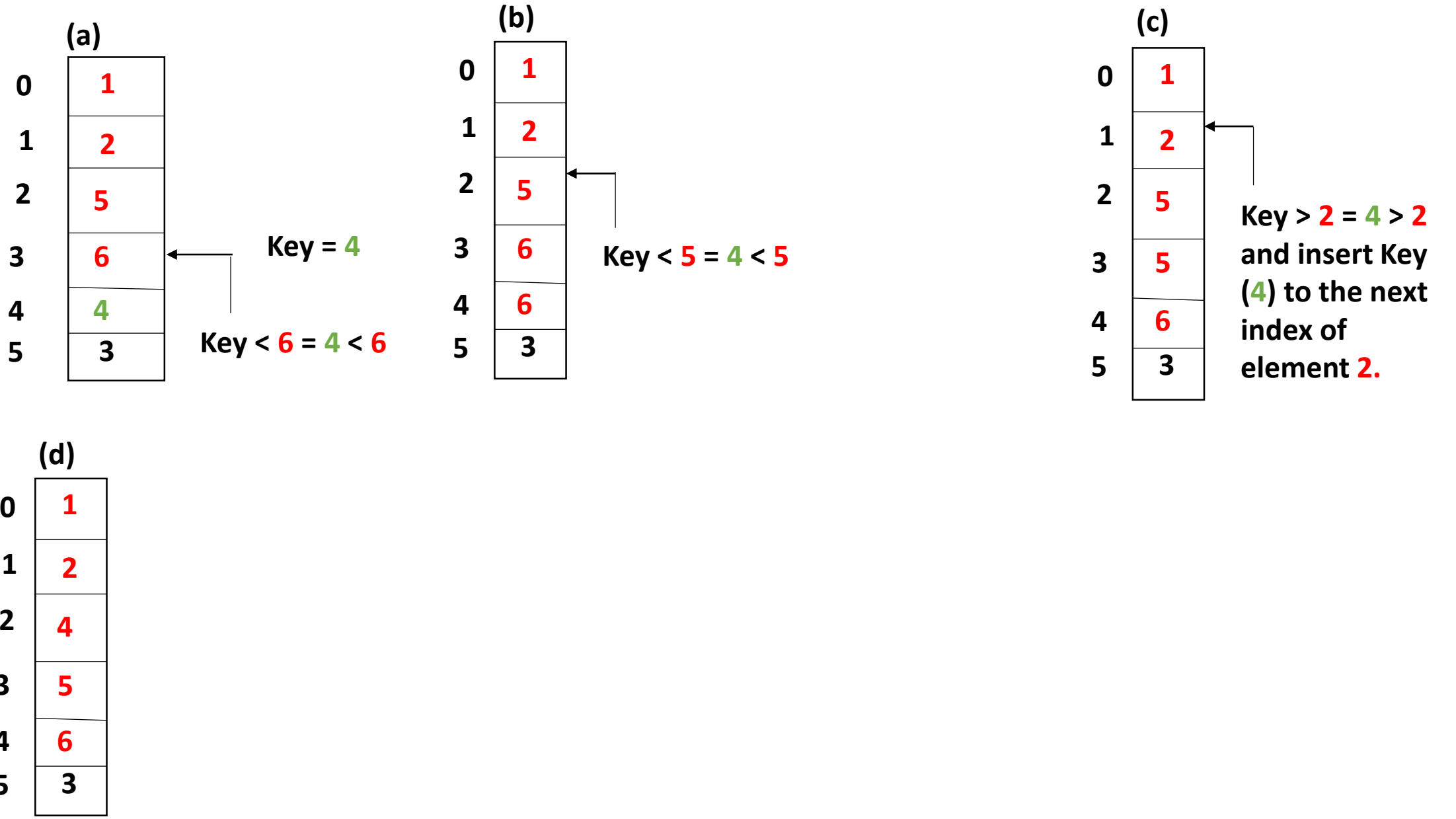
Cont...

Step 3:



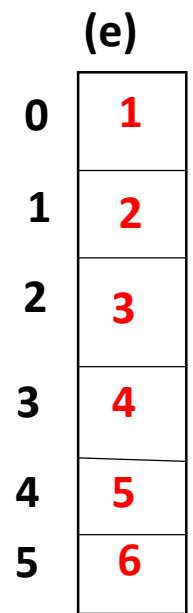
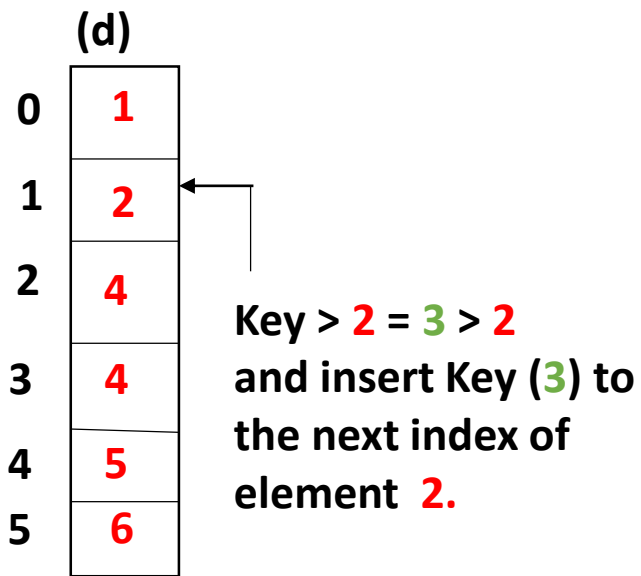
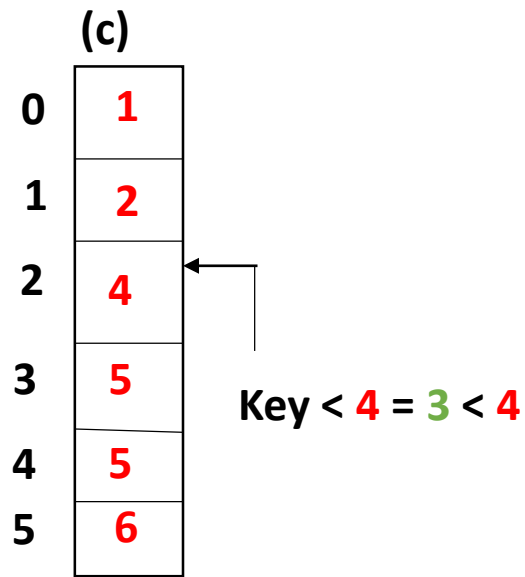
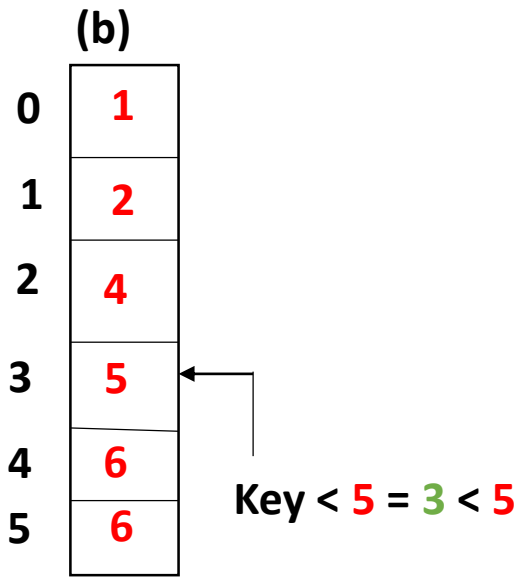
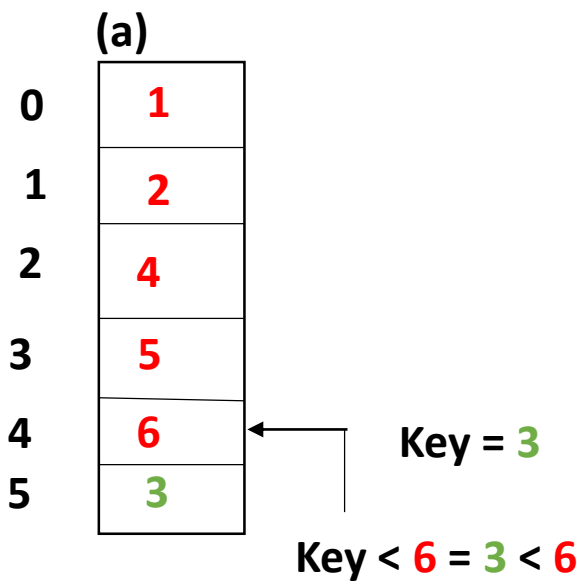
Cont...

Step 4:



Cont...

Step 5:



Program

```
#include<stdio.h>
void main()
{
    int a[6],n,i,j, key;
    printf("enter total no of elements");
    scanf("%d",&n);
    printf("enter elements");
    for(i=0;i<=n-1;i++)
    {
        scanf("%d",&a[i]);
    }
}
```

```
for (i = 1; i <= n-1; i++)
{
    key = a[i];
    j = i - 1;
    while (j >= 0 && key < a[j])
    {
        a[j + 1] = a[j];
        j = j - 1;
    }
    a[j + 1] = key;
}
for(i=0; i <= n-1; i++)
{
    printf("%d", a[i]);
}
}
```

Selection Sort

- The algorithm repeatedly selects the smallest (or largest) element from the unsorted portion of the list and swaps it with the first element of the unsorted portion.
- This process is repeated for the remaining unsorted portion of the list until the entire list is sorted.

Example

- Write steps during sorting the sequence 5, 1, 6, 2, 4 and 3 using selection sort.

Cont...

Ans:

	0	1	2	3	4	5
a	5	1	6	2	4	3

Cont...

Step 1:

(a)

a	0	5
	1	1
	2	6
	3	2
	4	4
	5	3

min = 5
loc = 0

← min > 1
5 > 1,
set
min = 1
loc = 1

(b)

a	0	5
	1	1
	2	6
	3	2
	4	4
	5	3

min = 1
loc = 1

← min < 6
1 < 6

(c)

a	0	5
	1	1
	2	6
	3	2
	4	4
	5	3

min = 1
loc = 1

← min < 2
1 < 2

(d)

a	0	5
	1	1
	2	6
	3	2
	4	4
	5	3

min = 1
loc = 1

← min < 4
1 < 4

(e)

a	0	5
	1	1
	2	6
	3	2
	4	4
	5	3

min = 1
loc = 1

← min < 3
1 < 3

so now min = 1 and loc=1, swap min element (1) with first element (5) of unsorted portion.

(f)

a	0	1
	1	5
	2	6
	3	2
	4	4
	5	3

Cont...

Step 2:

(a)

a	0	1
	1	5
	2	6
	3	2
	4	4
	5	3

min = 5
loc = 1
min < 6
5 < 6

(b)

a	0	1
	1	5
	2	6
	3	2
	4	4
	5	3

min = 5
loc = 1
min > 2
5 > 2,
set
min = 2
loc = 3

(c)

a	0	1
	1	5
	2	6
	3	2
	4	4
	5	3

min = 2
loc = 3
min < 4
2 < 4

(d)

a	0	1
	1	5
	2	6
	3	2
	4	4
	5	3

min = 2
loc = 3
min < 3
2 < 3

so now min = 2 and swap min element (2)
with first element (5) of unsorted portion.

(e)

0	1
1	2
2	6
3	5
4	4
5	3

Cont...

Step 3:

a

(a)

0	1
1	2
2	6
3	5
4	4
5	3

min = 6

loc = 2

min > 5

6 > 5,

set

min = 5

loc = 3

a

(b)

0	1
1	2
2	6
3	5
4	4
5	3

min = 5

loc = 3

min > 4

5 > 4,

set

min = 4

loc = 4

a

(c)

0	1
1	2
2	6
3	5
4	4
5	3

min = 4

loc = 4

min > 3

4 > 3,

set

min = 3

loc = 5

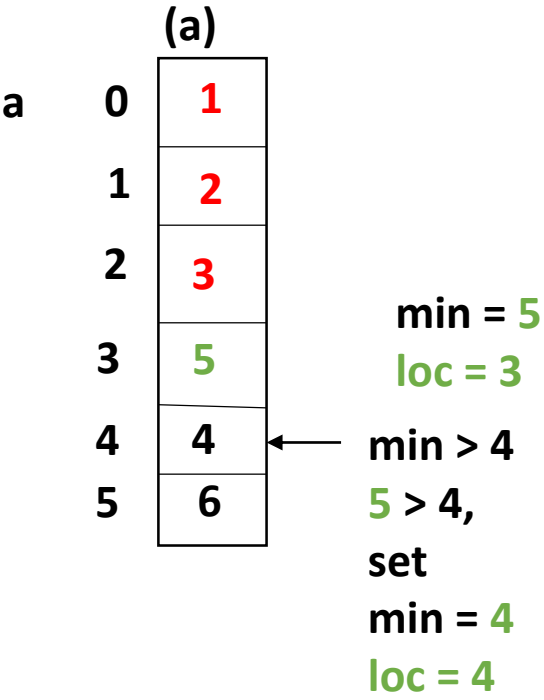
so now min = 3 and swap min element (3) with first element (6) of unsorted portion.

(d)

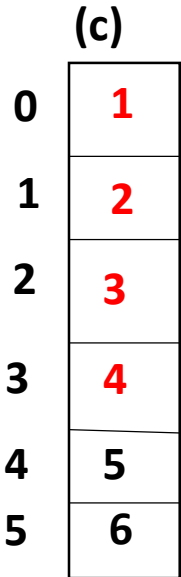
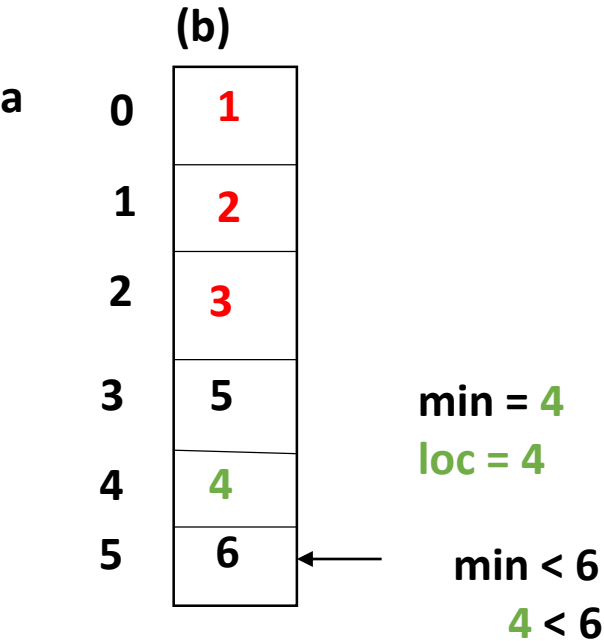
0	1
1	2
2	3
3	5
4	4
5	6

Cont...

Step 4:

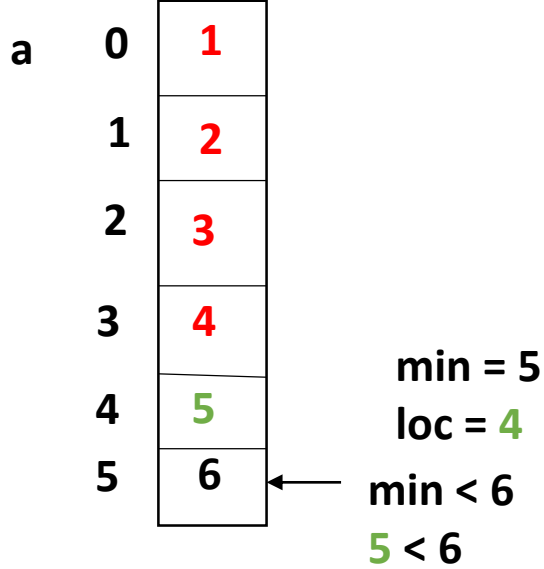


so now min = 4 and swap min element (4)
with first element (5) of unsorted portion.

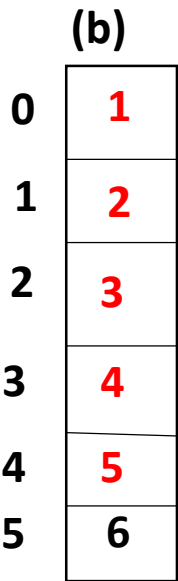


Cont...

Step 5: (a)



so now min = 5 and swap min element (5)
with first element (5) of unsorted portion.



Program

```
#include<stdio.h>
void main()
{
    int a[6], n, i, j, min, temp;
    printf("enter total no of elements");
    scanf("%d",&n);
    printf("enter elements");
    for(i=0;i<=n-1;i++)
    {
        scanf("%d",&a[i]);
    }
}
```

```
for (i = 0; i <= n-2; i++)
{
    min = a[i];
    loc = I;
    for (j = i+1; j <= n-1; j++)
    {
        if(min > a[j])
        {
            min = a[j];
            loc = j;
        }
    }
    a[loc] = a[i];
    a[i] = min;
}
for(i=0; i <= n-1; i++)
{
    printf("%d", a[i]);
}
}
```